

Questions before we move to objects?

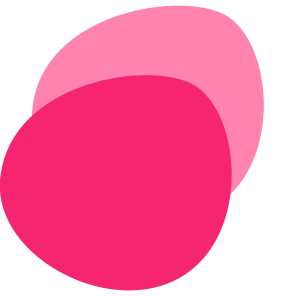
Remember: Practice is key to mastering arrays!



02

Objects

The Problem: Too Many Variables



Imagine tracking information about a user:

```
var userName = "Josh";  
var userAge = 28;  
var userEmail = "josh@example.com";  
var userCity = "Oakland";  
var userFavColor = "green";  
var userInstrument = "saxophone";
```

Too Many Variables

6 separate variables for ONE user

Doesn't Scale

What about user 2, 3, 4...?

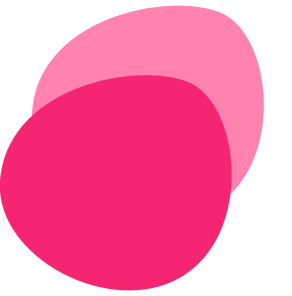
Hard to Pass

Difficult to pass related data together

Maintenance Nightmare

Difficult to organize and maintain

Could Arrays Help?



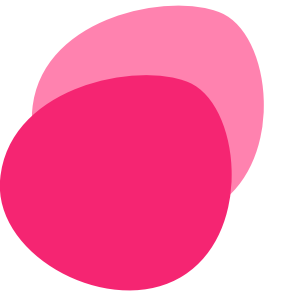
Attempt with an array:

```
var user = ["Josh", 28,  
  "josh@example.com",  
  "Oakland", "green",  
  "saxophone"];
```

This creates new problems:

```
// What does index 3 represent?
```

Objects: The Solution



Objects store data as **key-value pairs** with descriptive labels.



Name: Josh



Age: 28

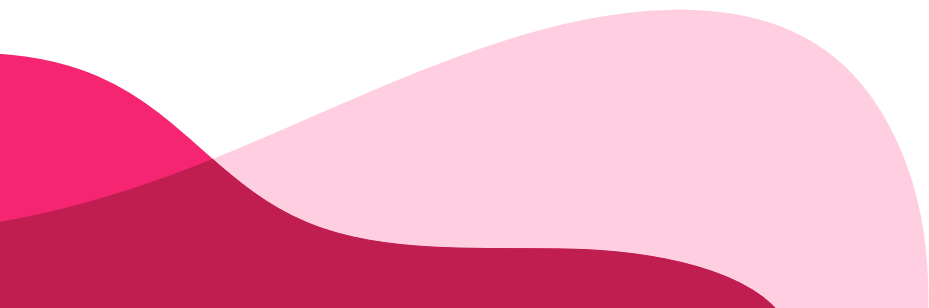


Email:
josh@example
.com



City: Oakland

Each piece of information has a **label** (key) and a **value**.



Creating Objects

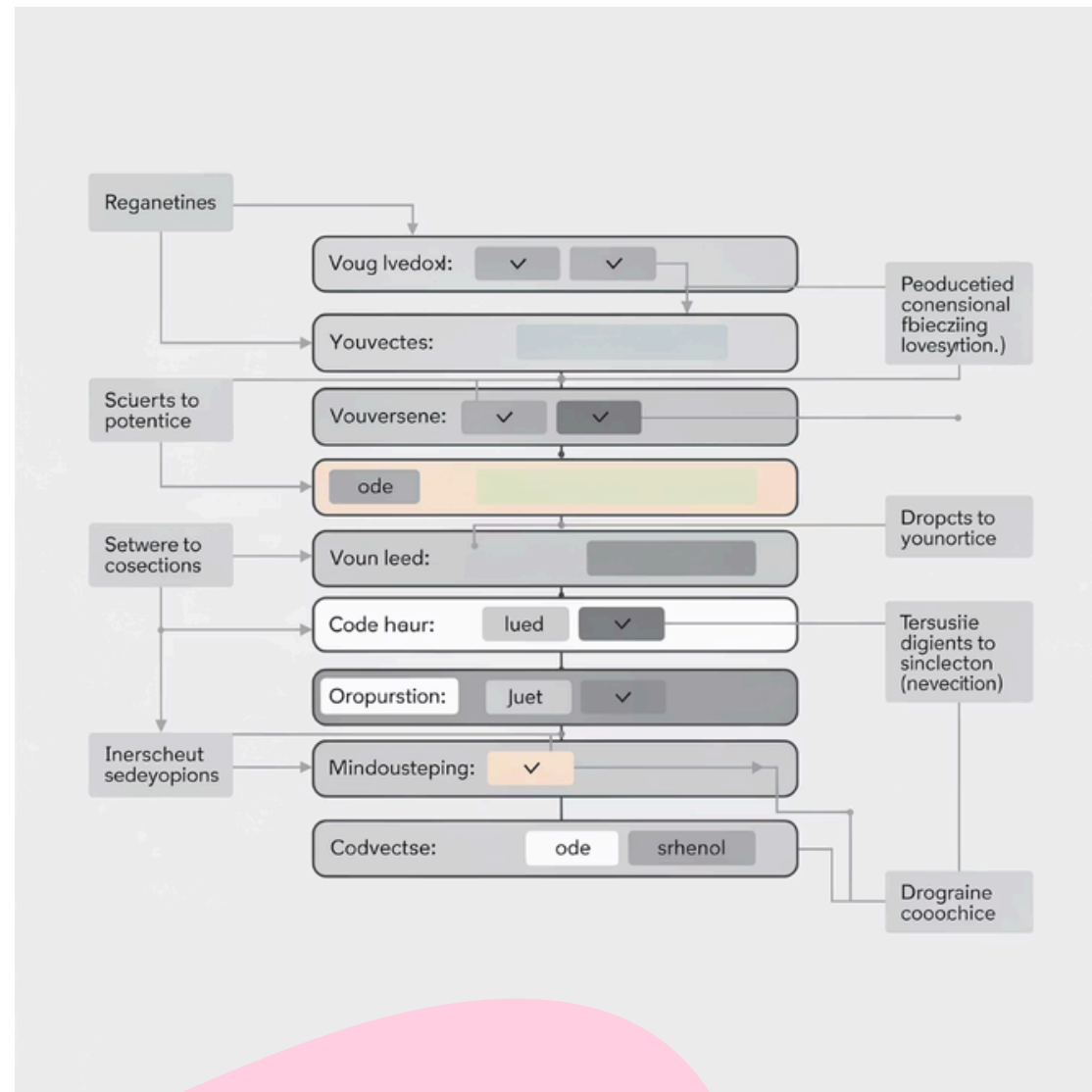


Syntax:

```
var objectName = {  
  key1: value1,  
  key2: value2,  
  key3: value3  
};
```

Real example:

```
var user = {  
  name: "Josh",  
  age: 28,  
  email: "josh@example.com",  
  city: "Oakland",  
  favColor: "green",  
  instrument: "saxophone"  
};
```



- 0 1 Use curly braces { }
- 0 2 Keys (properties) are like labels
- 0 3 Use colon : between key and value
- 0 4 Separate pairs with commas
- 0 5 Each key-value pair on its own line (best practice)

Object Syntax Breakdown

```
var user = {  
  firstName: "Josh",  
  lastName: "Lehman",  
  email: "jl@me.com"  
};
```



{ } Curly Braces

Define the object boundaries



: Colon

Separates key from value



, Comma

Separates key-value pairs



firstName - Key

Property name (no quotes needed)



"Josh" - Value

Any data type allowed



No Trailing Comma

After the last pair

Good vs Bad Object Formatting

✗ BAD - Hard to read:

```
var bad = {a: 1, b: 2, c: 3, d: 4};
```



✓ GOOD - Easy to read:

```
var good = {  
  a: 1,  
  b: 2,  
  c: 3,  
  d: 4  
};
```



Meaningful Key Names

Trailing Comma Optional
(Modern JS allows it)

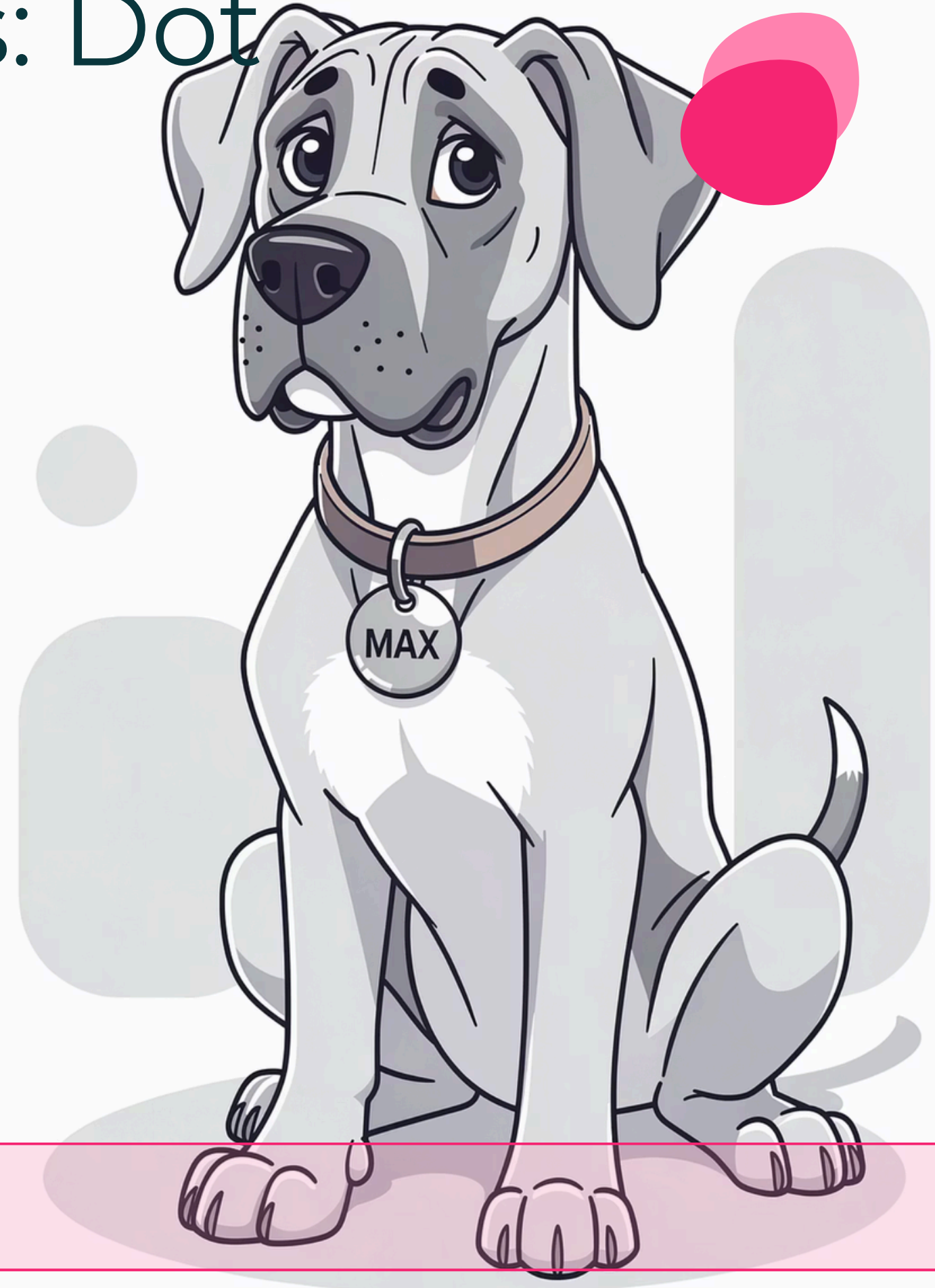
Consistent Indentation

One Property Per Line

Accessing Object Properties: Dot Notation

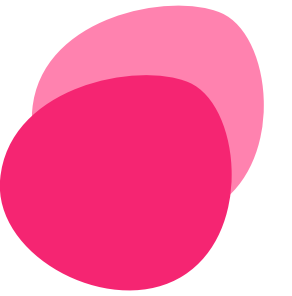
The **most common** way to access properties:

```
var dog = {  
  name: "Scooby Doo",  
  age: 7,  
  breed: "Great Dane"  
};  
  
// Access properties  
dog.name;    // "Scooby Doo"  
dog.age;     // 7  
dog.breed;   // "Great Dane"  
  
// Use in expressions  
console.log("My dog " + dog.name + " is a " + dog.breed);  
// Output: "My dog Scooby Doo is a Great Dane"  
  
// Store in variables  
var dogAge = dog.age;  
console.log(dogAge); // 7
```



Pattern: `objectName.propertyName`

Accessing Object Properties: Bracket Notation



Alternative syntax using square brackets:

```
var dog = {  
  name: "Scooby Doo",  
  age: 7,  
  breed: "Great Dane"  
};  
  
// Access with brackets (use strings!)  
dog["name"];    // "Scooby Doo"  
dog["age"];     // 7  
dog["breed"];   // "Great Dane"
```

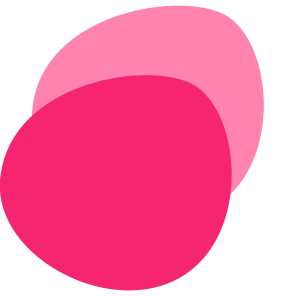
Pattern

```
objectName["propertyName"]
```

Important Note

Property name must be a **string** in brackets

Dot vs Bracket Notation: When to Use Each?



Use DOT notation (most common)

```
user.name;  
user.age;  
user.email;
```



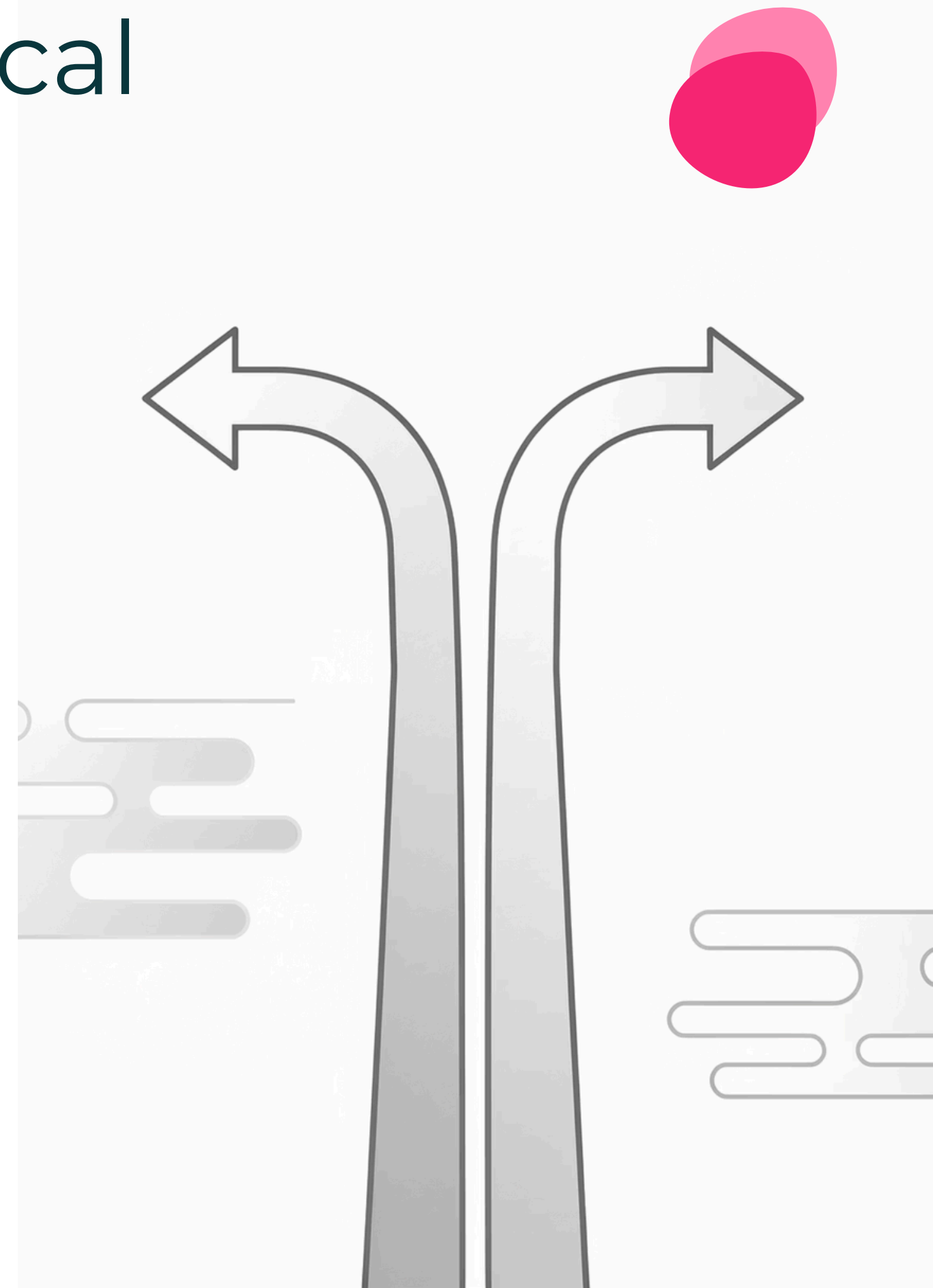
Use BRACKET notation when...

1. Property name is in a variable:

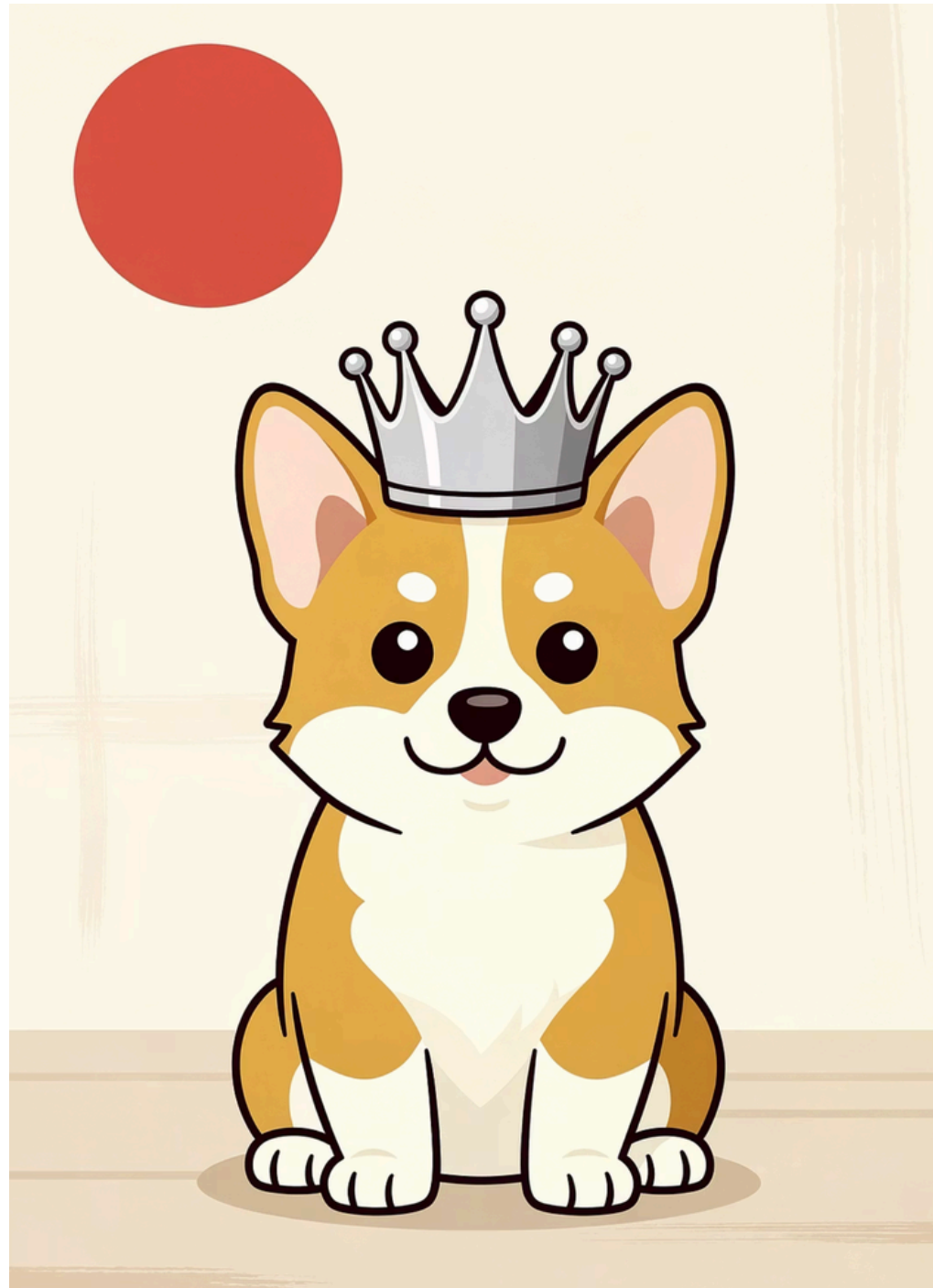
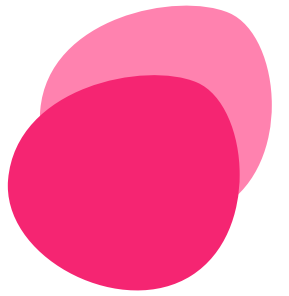
```
var dog = { name: "Rex", age: 5 };  
var prop = "name";  
  
dog[prop]; // "Rex"
```

Why Two Notations? A Critical Example

```
var dog = {  
  name: "Scooby Doo",  
  age: 7,  
  breed: "Great Dane"  
};  
  
var someProperty = "age";  
  
// What happens?  
dog[someProperty]; // 7
```



Modifying Object Properties

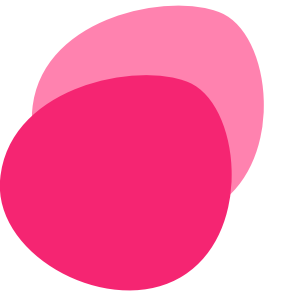


Change existing properties using assignment:

```
var puppy = {  
  name: "Gatsby",  
  age: 1,  
  breed: "Corgi"  
};  
  
// Modify with dot notation  
puppy.name = "Gatsby The Great";  
puppy.age = 2;  
  
// Modify with bracket notation  
puppy["breed"] = "Pembroke Welsh Corgi";  
  
console.log(puppy);  
// {  
//   name: "Gatsby The Great",  
//   age: 2,  
//   breed: "Pembroke Welsh Corgi"  
// }
```

Both notations work for modification!

Adding New Properties

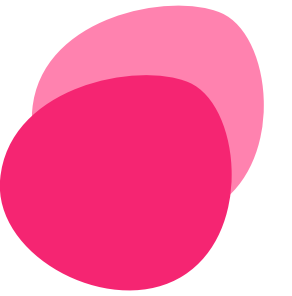


Create new properties on existing objects:

```
var puppy = {  
  name: "Gatsby",  
  age: 1,  
  breed: "Corgi"  
};  
  
// Add new properties  
puppy.biochipped = true;  
puppy.color = "brown";  
puppy["vaccinated"] = true;  
  
console.log(puppy);  
// {  
//   name: "Gatsby",  
//   age: 1,  
//   breed: "Corgi",  
//   biochipped: true,  
//   color: "brown",  
//   vaccinated: true  
// }
```

JavaScript allows dynamic property addition!

Checking if Property Exists



```
var user = {
  name: "Alice",
  age: 25
};

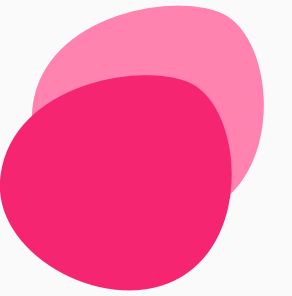
// Method 1: Check if undefined
if (user.email !== undefined) {
  console.log(user.email);
} else {
  console.log("Email not found");
}

// Method 2: Check truthiness (be careful with false/0)
if (user.name) {
  console.log("Name exists: " + user.name);
}

// Accessing non-existent property
user.phone; // undefined (not an error!)
```

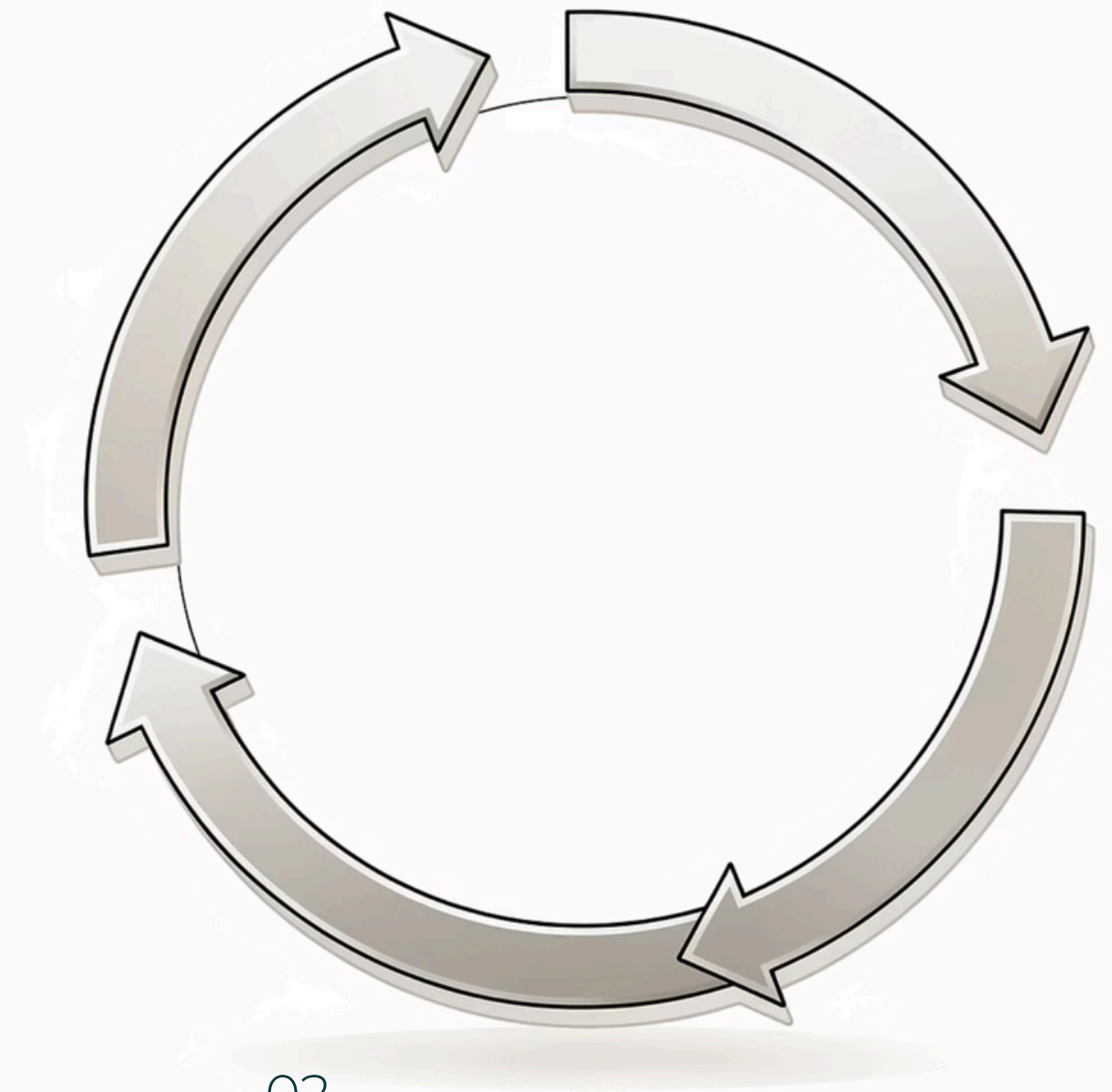
Note: Accessing non-existent properties returns undefined, not an error.

Looping Through Objects: for...in



Special loop for iterating over object properties:

```
var puppy = {  
  name: "Gatsby",  
  age: 1,  
  breed: "Corgi"  
};  
  
// Loop through all properties  
for (var key in puppy) {  
  console.log(key + ": " + puppy[key]);  
}  
  
// Output:  
// name: Gatsby  
// age: 1  
// breed: Corgi
```



01

key is a variable (can be any name)

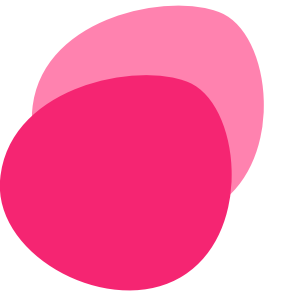
02

Holds the property name as a **string**

03

Use puppy[key] to access the value (must use brackets!)

Quick Reference: Objects



Looping Through Objects

```
// Loop
for (var key in obj) {
  console.log(key, obj[key]);
}
```

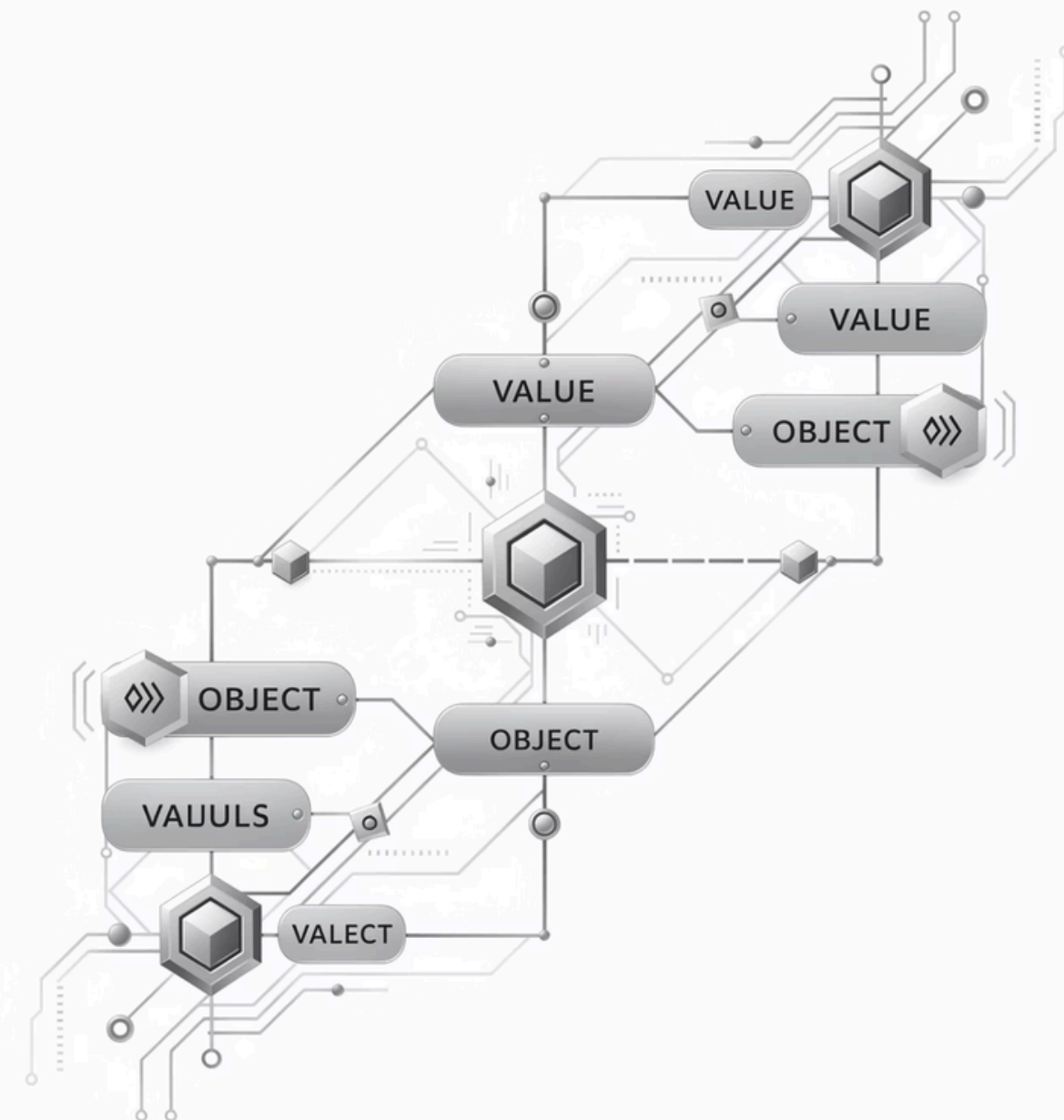
Creating & Accessing Objects

```
// Create
var obj = {
  key1: "value1",
  key2: 42
};

// Access
obj.key1      // Dot notation
obj["key1"]   // Bracket notation

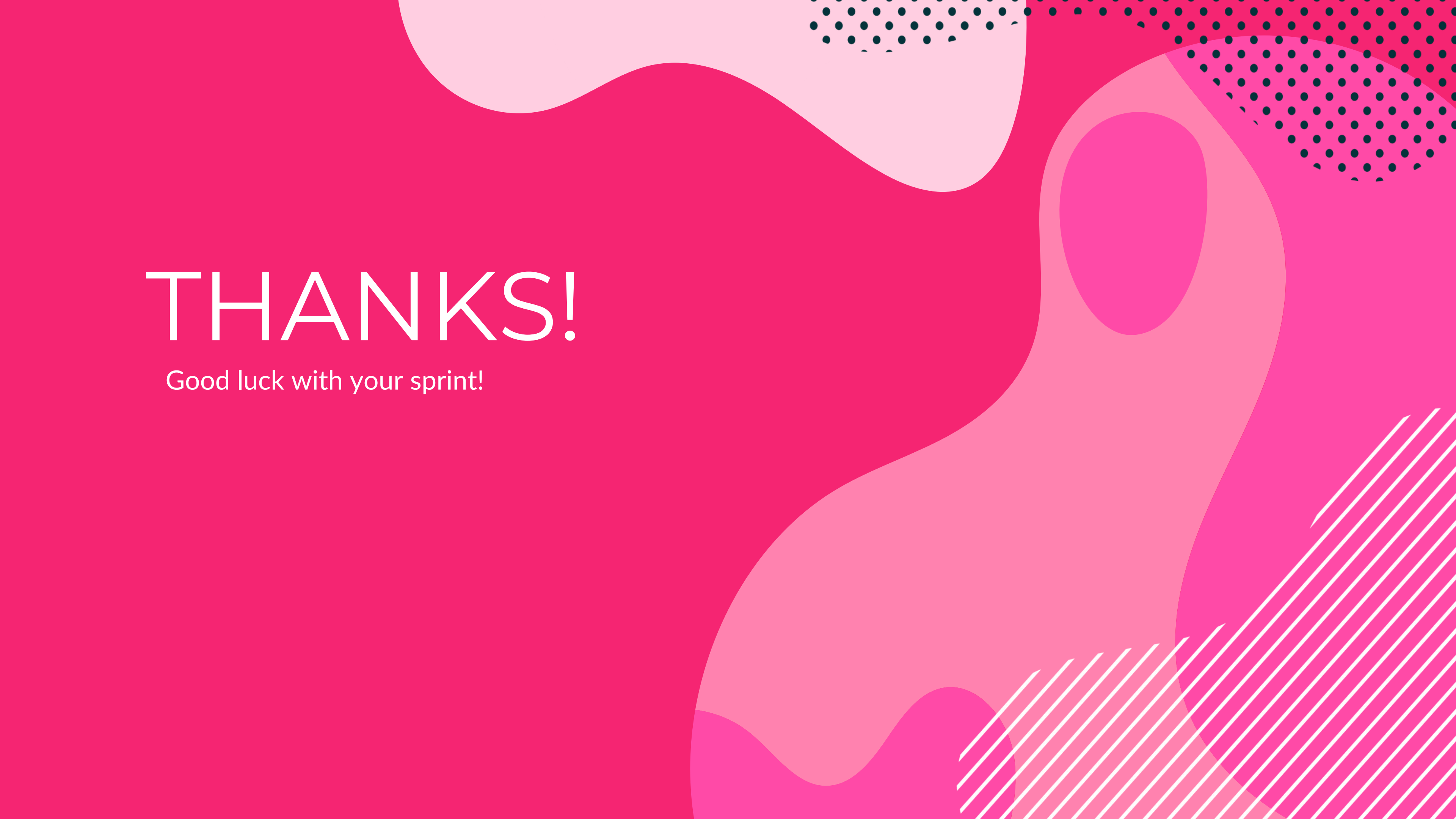
// Modify
obj.key1 = "new"

// Add
obj.key3 = true
```



Questions?





THANKS!

Good luck with your sprint!